TECHNICAL DOCUMENTATION & ENGINEERING STANDARDS

Enterprise Git Version Control & Branching Workflow

Trunk-Based Development, Conventional Commits, Pull Request SLAs, and Release Tags

System: Git 2.40+ / GitHub Enterprise | **Version:** 3.5 | **Owner:** DevOps & Release Engineering | **Source Reference:** Official Git Documentation

1. Branching Strategy: Trunk-Based Development

Apex practices Trunk-Based Development with short-lived feature branches to maximize integration velocity and minimize merge debt:

- `main`: The canonical production-deployable trunk. Direct pushes and force-pushes are strictly locked.
- `feature/<TICKET>-<description>`: Feature branches branched from latest `main` (lifespan < 48 hours).
- `bugfix/<TICKET>-<description>`: Standard defect remediation.
- `hotfix/<TICKET>-<description>`: Critical production patch branching directly from `main` tag.

2. Conventional Commit Messages Specification

Commit messages must adhere to the Conventional Commits specification to facilitate automated changelog generation and semantic versioning:

```
# Commit Message Format
<type>(<optional scope>): <description> [#TICKET-ID]

# Allowed Types:
# feat:     A new feature for the user
# fix:      A bug fix
# docs:     Documentation changes only
# refactor: Code change that neither fixes a bug nor adds a feature
# test:     Adding missing tests or correcting existing tests
# chore:    Build tasks, dependency bumps, or tool configuration

# Example:
feat(rag): implement hybrid sparse-dense vector retrieval [#APEX-1042]
```

3. Pull Request (PR) Lifecycle and Code Review Protocol

Phase	Required Criteria	Enforcement Mechanism
PR Creation	PR description completed using standard template; linked Jira ticket; <400 lines changed.	GitHub PR Template & DangerJS bot check.
Automated CI	Unit tests passing (100%); coverage ≥85%; Ruff linting green; Snyk security scan clean.	GitHub Actions Branch Protection Rules.
Peer Review	Minimum of two (2) approvals from team members (at least one Senior/Staff engineer).	GitHub CODEOWNERS requirement.
Merge Strategy		Repository merge enforcement settings.

Phase	Required Criteria	Enforcement Mechanism
	Squash and Merge to preserve linear git history on `main`.	

4. Rebasing, Merge Conflicts, and Semantic Version Tagging

```
# Keep your local feature branch up to date with trunk via rebase
git checkout main && git pull origin main
git checkout feature/APEX-1042-hybrid-retrieval
git rebase main

# Semantic Release Tagging on main trunk
git tag -a v2.4.0 -m "Release v2.4.0: Knowledge Assistant RAG v2"
git push origin v2.4.0
```